PROJECT GUIDE · DVP

Supply Chain Intelligence Dashboard

How it works, where everything lives, and how a request travels from a click in the browser through FastAPI, pandas and Groq and back to the screen.

1 What the project does

2 Architecture map

3 File-by-file map

4 Flow: from CSV to dashboard

5 Flow: asking the copilot

6 Flow: proactive insights

7 Flow: the Scenario Lab

8 The analytics, explained

9 Frontend internals

10 Lessons from the bugs we fixed

11 API reference

12 Run, configure, troubleshoot

13 Extend it, and a learning path

Written for ria · 12 October 2026 · Stack: FastAPI, pandas, NumPy, Groq (openai/gpt-oss-120b), React 18, Vite, Recharts · Reference dataset: fmcg_clean.csv (186,892 rows, Jan 2022 to Dec 2024)

1. What the project does

The dashboard turns one sales-and-inventory CSV into answers to three questions: what is selling, where stock will run out, and what happens if the supply chain is disrupted.

You load a CSV. The backend cleans it with pandas and computes every metric. The browser draws those metrics as charts. Groq, a hosted large language model, then explains the numbers in plain English. The model never invents the numbers: it only receives ones the backend already computed.

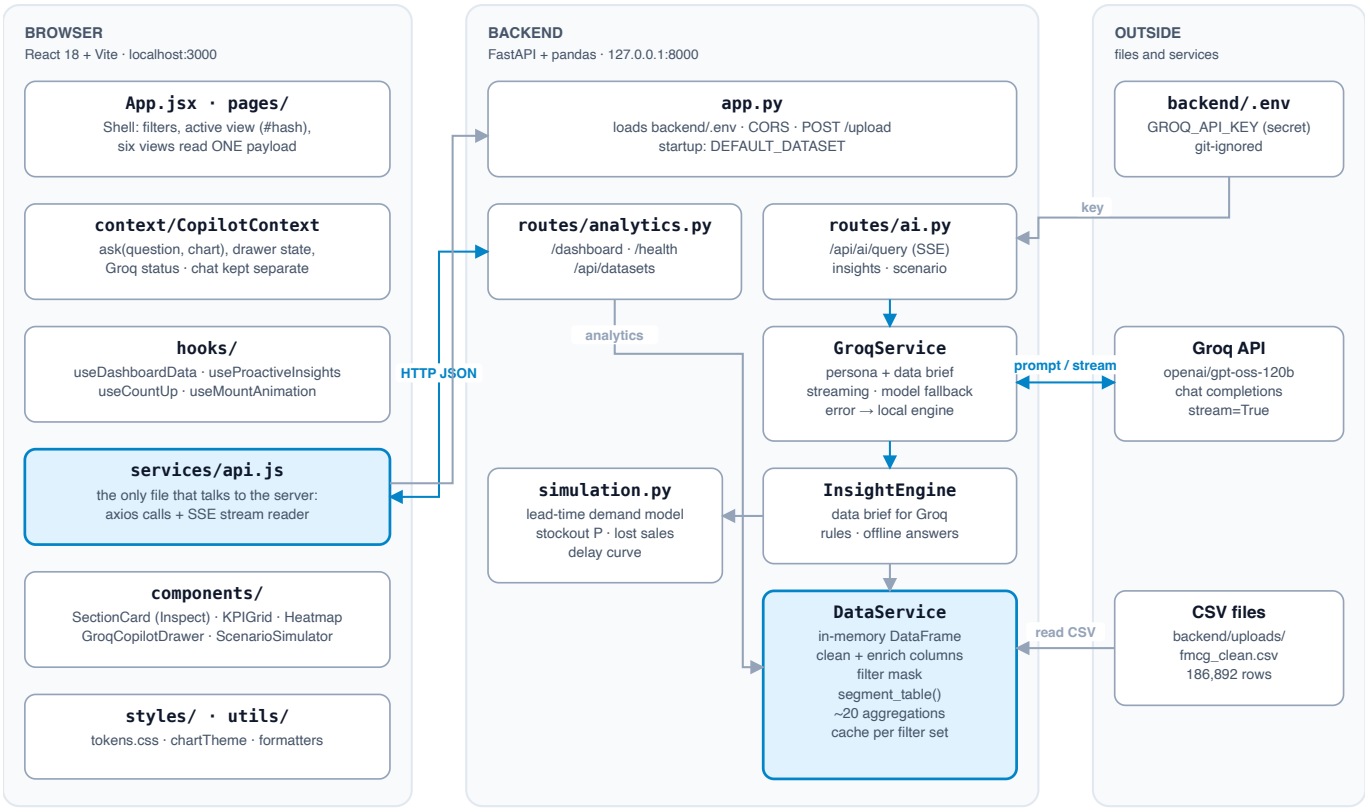
Each CSV row is one sale record. Six columns are required: `date`, `units_sold`, `price_unit`, `delivery_days`, `delivered_qty`, `stock_available`. Optional columns (`region`, `channel`, `category`, `sku`, `promotion_flag`) unlock the breakdowns.

View	Question it answers	What you see
Overview	How healthy is the business right now?	Five KPI cards with deltas and sparklines, revenue trajectory vs moving average, proactive signals, top stock deficits, recommended actions
Demand & Sales	Where does revenue come from, and does price move volume?	Region × channel stacked revenue, channel efficiency matrix, price-elasticity scatter with fitted curve, category bars
Operations	Are deliveries late, and which items are short?	Lead-time histogram with normal curve, lead time by region, stock vs velocity quadrant, replenishment queue
Seasonality	When and where does demand peak, and what comes next?	Season × region heatmap, 7-period forecast with 95% band, seasonal volume and lead time
Scenario Lab	What if suppliers slip or demand surges?	Sliders with live lost sales, stockout probability, fill rate, holding cost, delay curve
Intelligence Hub	What should I do about it?	AI insight pills, one-click prompts, demand split, recommendations

Two features run across every view. The **Copilot** (⌘K or Ctrl+K, or the floating pill) is a chat drawer. The **Inspect** button on every chart sends that chart's data to the copilot with a tailored question. Four filters (region, channel, category, season) apply to every view and to everything the AI sees.

2. Architecture map

The backend computes every number; Groq only puts them into words



Blue boxes are the two files to understand first: `services/api.js` is the browser's only door to the server, and `DataService` is where every number starts.

Reading the map, box by box

Part	What happens there
Browser	React draws everything. `App.jsx` holds the filters and the active view; the views are plain functions of one data object. No business logic lives here: the browser formats and draws.
api.js	Every request leaves through this file. It uses axios for normal JSON calls and a small hand-written reader for the copilot's streamed answers.
app.py	Starts the server, loads `backend/.env` (where your Groq key lives), allows the browser's origin (CORS), handles uploads, and can preload a dataset at start-up.
routes/	Thin HTTP layer. Each function reads query parameters or a JSON body, calls a service, and turns Python errors into HTTP errors. `analytics.py` serves the dashboard; `ai.py` serves the copilot, insights and scenarios.
DataService	Owns the dataset: cleaning, filtering, every aggregation and the cache. Everything else asks it for data.
InsightEngine	Turns DataService output into words: the dense data brief sent to Groq, the rule-based insight scan, and the offline answers used when Groq is unavailable.
simulation.py	The what-if inventory model behind the Scenario Lab. Pure maths, no AI.
GroqService	The only file that talks to Groq: builds the prompt, streams the answer, retries on a retired model, and falls back to InsightEngine on any error.

Outside	The CSV files on disk, the Groq API on the internet, and the secret key in `backend/.env`, which git ignores.

> **The one idea to remember**
>
> The backend is the single source of truth. The browser never computes a metric, and Groq never computes a number. That is why the dashboard still works with no key: only the wording changes.

3. File-by-file map

About 1,900 lines of Python and 3,900 lines of JavaScript and CSS. Numbers in the last column are line numbers, so you can jump straight to a function in your editor.

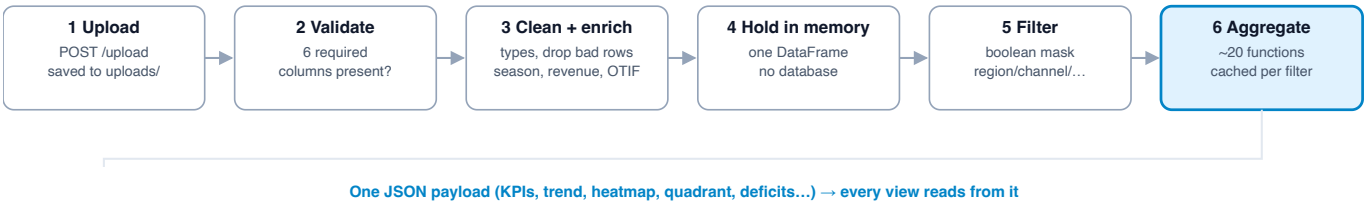
File or folder	Lines	Responsibility	Key functions (line)
`backend/app.py`	69	Server entry point: .env loading, CORS, POST /upload with filename sanitising, DEFAULT_DATASET at startup	`load_default_dataset (43)`, `upload_dataset (57)`
`backend/routes/analytics.py`	92	Dashboard and dataset endpoints; legacy /sales, /seasonality…	`dashboard (25)`, `list_datasets (75)`, `load_dataset (84)`
`backend/routes/ai.py`	81	AI endpoints; wraps the copilot stream as Server-Sent Events	`ai_query (34)`, `proactive_insights (59)`, `scenario_simulate (72)`
`backend/models/schemas.py`	43	Pydantic request models; malformed input gets a clear 422	`AIQueryRequest`, `ScenarioRequest`
`backend/services/data_service.py`	766	Load, clean, filter, aggregate, cache. The heart of the project	`_preprocess (84)`, `filtered (147)`, `_build_payload (199)`, `segment_table (304)`, `_kpis (369)`, `_forecast_series (467)`
`backend/services/insight_engine.py`	472	Data brief for Groq, proactive rules, offline answers	`build_context (44)`, `proactive_insights (101)`, `offline_answer (235)`
`backend/services/simulation.py`	146	What-if inventory model	`_evaluate (50)`, `simulate (99)`
`backend/services/groq_service.py`	261	Groq client, prompts, streaming, model fallback, error messages	`_create (111)`, `_messages (134)`, `stream_query (149)`, `proactive_insights (200)`, `scenario (235)`
`frontend/src/App.jsx`	189	Shell: filters, active view from #hash, loading / empty / error states, error boundary	`App`, `Shell`, `View`
`frontend/src/context/CopilotContext.jsx`	132	Copilot state split in two contexts (actions vs chat transcript)	`CopilotProvider`, `useCopilot`, `useCopilotChat`
`frontend/src/services/api.js`	132	All HTTP calls; SSE parser for streaming	`fetchDashboard`, `streamAIQuery`, `simulateScenario`

`frontend/src/hooks/`	196	Data fetching and small behaviours	`useDashboardData`, `useProactiveInsights`, `useCountUp`, `useMountAnimation`, `useHotkey`
`frontend/src/components/`	1415	Reusable UI: cards, KPI grid, tooltip, heatmap, drawer, simulator, markdown renderer	`SectionCard`, `KPIGrid`, `GroqCopilotDrawer`, `ScenarioSimulator`
`frontend/src/pages/`	784	The six views	`DashboardPage`, `SalesPage`, `OperationsPage`, `SeasonalPage`, `SimulatorPage`, `AIPage`
`frontend/src/styles/tokens.css` + `App.css`	903	Design system: colours, type, spacing, motion, every component style	`CSS variables in :root`
`frontend/src/utils/`	130	Number formatting and the shared chart theme	`formatCurrency`, `formatKpi`, `COLORS`, `axisProps`

Also in the repo: `backend/uploads/` (CSV files you can load), `backend/.env.example` (template; the real key goes in `backend/.env`), `docs/research/DET.ipynb` (the original exploration notebook), and this guide under `docs/`.

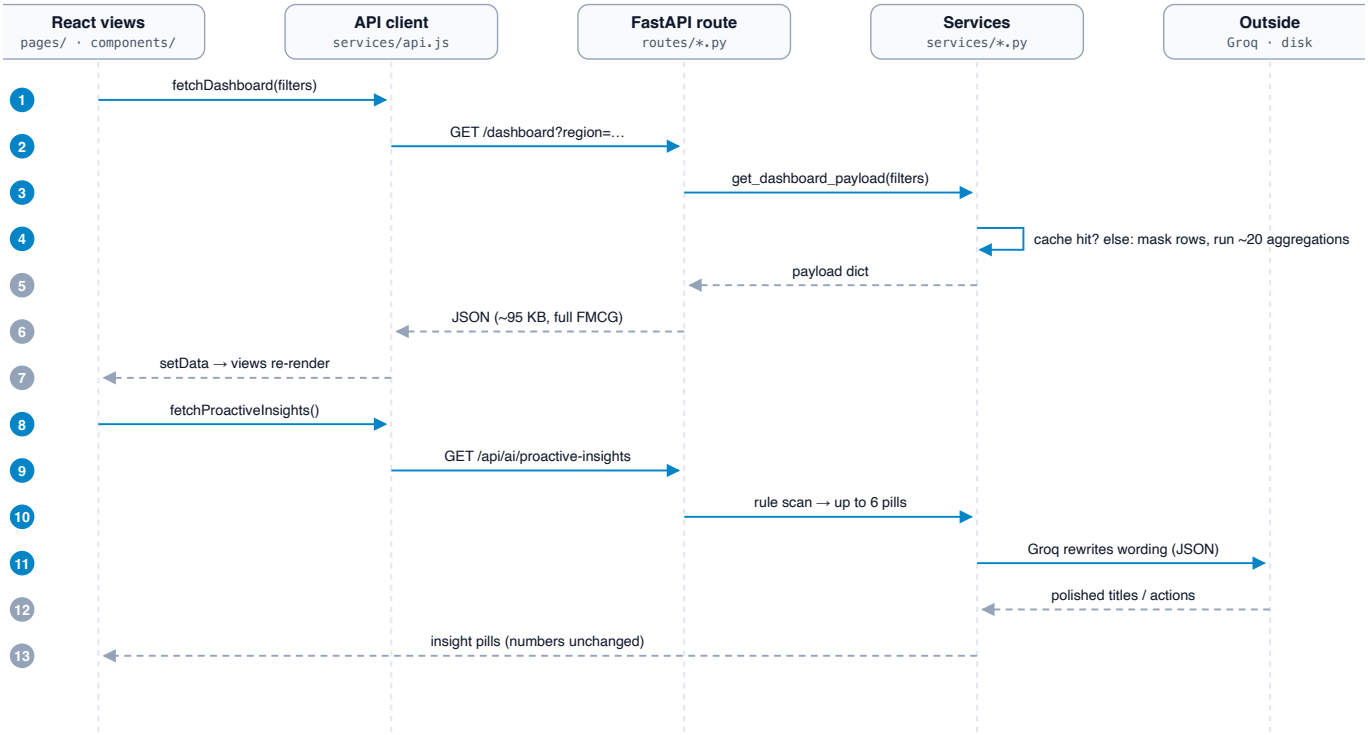
4. Flow: from CSV to dashboard

From a CSV row to a number on screen



Steps 1 to 4 run once per upload; steps 5 and 6 run on every filter change.

Flow 1 · Opening the dashboard (and every filter change)



Read top to bottom. Solid arrows are requests, dashed arrows are responses.

What each step does, in code

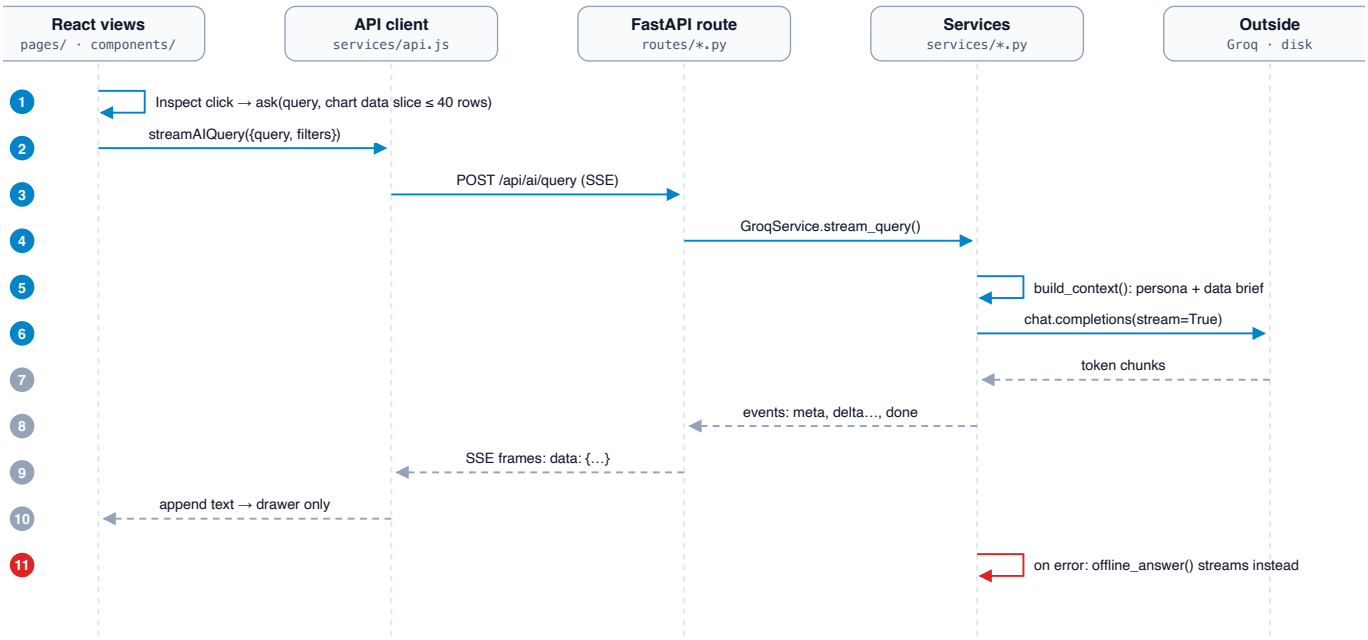
1. **Upload.** `POST /upload` saves the file under `backend/uploads/`. The filename is reduced to its last part, so a name like `../../app.py` cannot escape the folder.
2. **Validate.** `process_upload()` rejects the file with a 400 listing any missing required column.
3. **Clean and enrich.** `_preprocess()` converts types, drops rows with missing values or non-positive price, and adds: `season` (from the month), `revenue` = units × price, `on_time` (≤ 3 days), `in_full` (delivered ≥ sold), and `segment` (SKU · region).
4. **Hold in memory.** The DataFrame lives on the single `data_service` object. There is no database: a restart forgets it unless `DEFAULT_DATASET` is set.
5. **Filter.** `filtered()` builds a True/False mask from the query string. `channel=Retail,Discount` selects both.
6. **Aggregate and cache.** `_build_payload()` runs about 20 small functions and stores the result under a key made from the filters, so revisiting a filter is instant. A new upload clears the cache.

Why one big payload?

One request gives every view what it needs, so switching views is instant and all charts always agree with each other. Only the copilot, the insight scan and the Scenario Lab make their own requests, because they are slower or interactive.

5. Flow: asking the copilot

Flow 2 · Asking the copilot (typed question or Inspect button)



The red dashed step is the fallback path: if Groq fails at any point, the local engine streams an answer through the same pipe.

Context hydration: what Groq actually receives

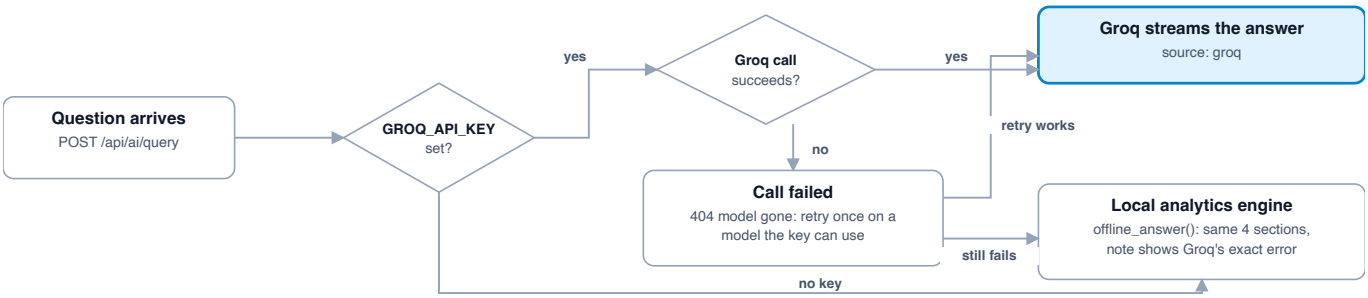
The model never sees your CSV. `_messages()` sends four things:

1. **Persona**: "You are an elite Chief Supply Chain Officer and Quantitative Operations Analyst…", which keeps answers numeric and concrete.
2. **Answer format**: four required sections (Executive answer, Root cause hypothesis, Tactical actions with dollar impact, Metric citations) and a glossary of velocity, cover and reorder point.
3. **Data brief** from `build_context()`: about 15 dense lines with dataset, rows, date range and filters; KPIs; lead time by region, channel and season; the 5 tightest SKU-locations; the seasonality index grid; revenue by channel, region and category; elasticity; quadrant counts; the forecast. For an Inspect click, up to 40 rows of that chart are appended.
4. **History**: the last 8 exchanges, so follow-ups like "and in the South?" work.

Streaming, step by step

The route returns `text/event-stream`. Each event is one line, `data: {json}`, followed by a blank line. The first event, `meta`, says who is answering; many `delta` events carry text; `done` ends it. In the browser, `streamAIQuery()` reads the response body chunk by chunk, splits on blank lines, and calls back for each event. The drawer appends the text, which gives the typewriter effect.

Who answers your question: Groq if it can, the local engine otherwise



Every path ends in an answer. The note at the top of a fallback answer quotes Groq's exact error, for example an expired key.

6. Flow: proactive insights

Proactive insights run the opposite way to the copilot: Python rules find the problems, and Groq only rewrites the wording. `InsightEngine.proactive_insights()` checks, on the filtered data:

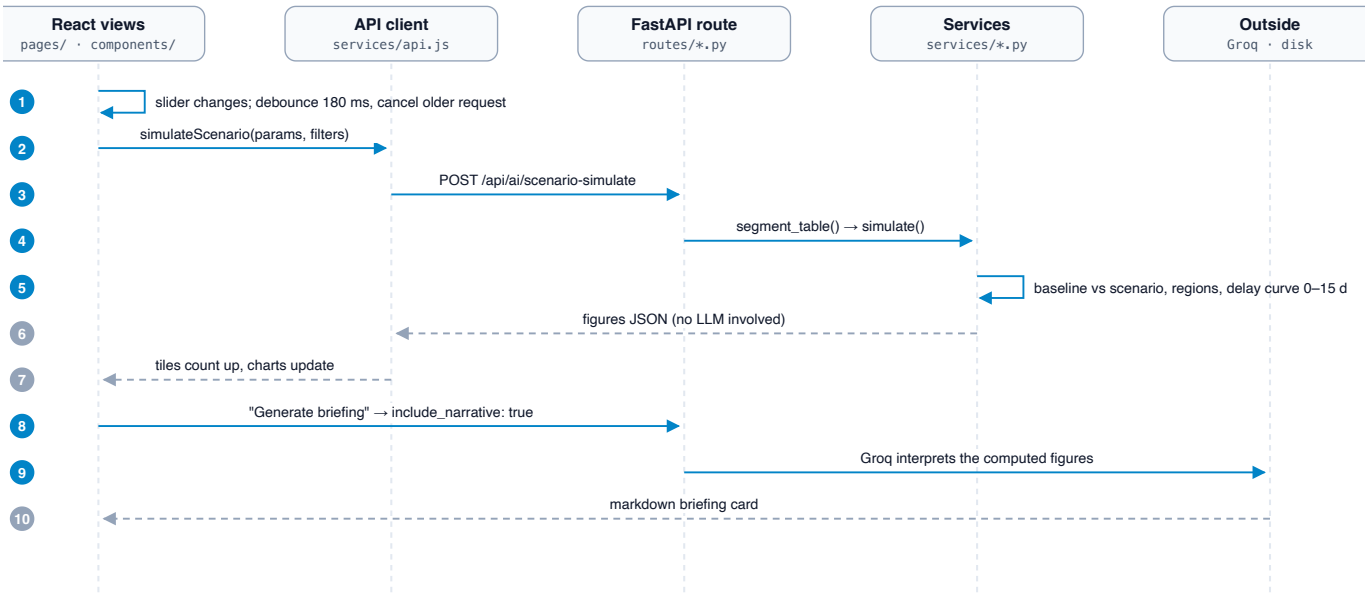
Check	Rule	Severity
Stockout hazard	Any SKU-location below its 95% reorder point, or whose cover is shorter than lead time + 7 days	CRITICAL / WARNING
Lead-time anomaly	Deliveries slower than mean + 2σ	CRITICAL if over 3% of orders
SLA breach	More than 20% of deliveries beyond the 3-day SLA	WARNING
Margin leak	Promoted orders earn less revenue per order than unpromoted ones	WARNING
Seasonal mismatch	The peak-demand season carries less stock per unit sold than average	WARNING / OPPORTUNITY
Channel gap	Best region × channel earns over 3% more per unit than the worst	OPPORTUNITY
Revenue momentum	Latest 28 days moved 5% or more vs the 28 before	WARNING / OPPORTUNITY

Hits are sorted by severity, then by estimated dollar impact, and the top 6 are kept. With a key set, `GroqService.proactive_insights()` asks Groq, in JSON mode, to tighten each title, detail and action while keeping ids, severities and numbers. If that call fails, the rule text is shown unchanged.

On `fmcg_clean.csv` the scan returns four pills: SLA breach rate 40.2% (PL-South slowest), 47 segments with thin cover, Monsoon under-stocked for its demand (7.07 units of stock per unit sold vs 8.0 average), and promotions that lift units 95%.

7. Flow: the Scenario Lab

Flow 3 · Moving a Scenario Lab slider



The figures come only from the maths below; Groq is involved only if you press Generate briefing.

The model, in five steps

1. **Shift the inputs** for every SKU-location: lead time L → (L + delay) × (1 + spike%); demand d → d × (1 + surge%) × (1 + price change)^{elasticity}; stock S → S × buffer.
2. **Lead-time demand** is treated as a normal distribution with mean μ = d·L and spread σ = $\sqrt{(L \cdot \sigma_d^2 + d^2 \cdot \sigma_L^2)}$. Both day-to-day demand and the lead time itself vary.
3. **Stockout probability** is the chance demand exceeds stock: P = 1 − Φ(z), with z = (S − μ) / σ.
4. **Units short per cycle** use the standard normal loss function below. Multiplying by 30 / L cycles gives lost units over 30 days; × price gives lost sales.
5. **Holding cost** = stock value × 25% a year × 30/365, so you can see what a bigger buffer costs.

$$E[\text{short}] = \sigma \cdot (\varphi(z) - z \cdot (1 - \Phi(z)))$$

On `fmcg_clean.csv`, adding 8 days of supplier delay moves 30-day lost sales from $119 to $101.5K and fill rate from 100% to 82.5%. The delay curve shows that losses stay near zero for a few days and then climb steeply, which is the shape you would expect when stock covers about 8 days of demand.

8. The analytics, explained

Most numbers come from one idea: treat each **segment** (one SKU in one region) as a small warehouse, and compare how many days its stock lasts with how long a refill takes.

Metric	How it is computed	How to read it	FMCG value
Total revenue	Σ units_sold × price_unit	Money in for the filtered range	$19.95M
Delta	Last 28 days vs the 28 before (halves for short files)	Getting better or worse?	+0.7%
Inventory turnover	units sold ÷ stock on hand × 365	Times a year stock sells through	46.1×/yr
OTIF rate	Share delivered within 3 days AND delivered ≥ sold	Service level; 3 days is set by `OTIF_SLA_DAYS`	59.8%
Velocity	Average daily units over the segment's last 7 days	How fast it sells now	24 to 63 units/day
Cover	stock ÷ velocity	Days until empty without a reorder	6 to 17 days
Need	average lead time + 7-day review cycle	Days until a refill could arrive	about 10 days
Reorder point	$d \cdot L + 1.65 \cdot \sqrt{(L \cdot \sigma_d^2 + d^2 \cdot \sigma_L^2)}$	Stock that gives a 95% chance of no stockout	155 to 348 units
Stockout risk index	Demand-weighted average of how far cover falls short of need (0–100)	0 = all covered, 100 = all empty before refill	8.3
Quadrant	Velocity and stock split at their medians	Critical Low, Fast Mover, Overstocked, Dead Stock	15 / 31 / 14 / 30
Price elasticity	Slope of log(units) on log(price)	–1 means +10% price loses about 10% volume	≈ 0 (synthetic data)
Seasonality index	Daily units in a season ÷ that region's average	1.16 = 16% above normal	Monsoon 1.16
Forecast	Holt's linear smoothing, band = $\pm 1.96 \cdot \sigma \cdot \sqrt{h}$	Next 7 periods with an honest range	MAPE 6.3%

> **Assumptions you can change**
>
> Four constants at the top of `data_service.py` set the business rules: `OTIF_SLA_DAYS = 3`, `REVIEW_PERIOD_DAYS = 7`, `SERVICE_Z = 1.65` (95% service level) and `VELOCITY_WINDOW_DAYS = 7`. OTIF is 59.8% only because 40% of FMCG deliveries take 4 or 5 days against the 3-day SLA.

Two details prevent misleading charts. Weekly series drop a final partial week, which otherwise looks like a demand collapse. The fitted elasticity is clamped between –3 and –0.1 inside the simulator so a noisy fit cannot produce absurd demand swings.

9. Frontend internals

How state flows

`App` owns the filters. It passes them to `useDashboardData`, which fetches the payload, and to `CopilotProvider`, so every AI request carries the current filters. Views receive `data` as a prop and render; they hold almost no state of their own.

Piece	Job	Detail worth knowing
`useDashboardData`	Fetch, refetch on filter change, upload, load sample	Numbers each request and ignores stale replies, so fast filter clicks never show old data. Skeletons appear only on the first load.
`CopilotContext` + `CopilotChatContext`	Copilot actions vs chat transcript	Split so streamed tokens re-render only the drawer, not the dashboard.
`SectionCard`	Every card; owns the Inspect button	`inspect={{ data, query }}` is all a chart needs to get AI help.
`useMountAnimation`	Charts animate once	Later updates swap in place instead of replaying the draw-in.
`useCountUp`	KPI numbers glide to new values	Ease-out over 0.9 s; disabled when the OS asks for reduced motion.
`ErrorBoundary`	Contains crashes	A broken view shows an error card with Try again instead of a blank page.
`Markdown`	Renders copilot answers	Dependency-free: headings, lists, tables, code, quotes, bold.

The design system

All styling flows from CSS variables in `styles/tokens.css`. Surfaces are obsidian (`#0B0F17`, `#111827`, `#161F30`) with 8%-white hairline borders. Four status colours carry meaning: emerald `#10B981` healthy, amber `#F59E0B` warning, coral `#EF4444` critical, ice blue `#38BDF8` primary. Text uses Inter with –0.015em tracking; every number uses JetBrains Mono with tabular figures so digits line up in columns. One spring curve (`--ease-spring`) drives hover lifts, view transitions and the drawer. Charts read the same palette from `utils/chartTheme.js`, and tooltips use a 12px backdrop blur.

Views live in the URL hash (`/#operations`), so a link opens the right view, and keys 1 to 6 switch between them.

10. Lessons from the bugs we fixed

Four problems came up after the first build. Each one teaches something general, so they are worth understanding.

Symptom	Cause	Fix	Lesson
"Connect Groq" stayed on after adding the key	The key was pasted into `backend/.env.example`; the app reads `backend/.env`	Moved the key to `.env` (git-ignored), restored the template	`.example` files are committed templates. Secrets go only in ignored files.
"Groq was unreachable (NotFoundError)"	Groq retired `llama-3.3-70b-versatile` for this key	Default `openai/gpt-oss-120b`; auto-retry on an available model; exact error shown	Hosted model names change. Ask the API what exists, and surface real error messages.
Page flickered, content vanished	Every streamed token re-rendered the whole app; Recharts replayed its draw-in on each new data array	Split the copilot context; animate charts on first mount only	In React, put fast-changing state where only its consumers subscribe.
"minimumFractionDigits is out of range"	The tooltip called `formatNumber(value, row)`, so the data row became the decimal count	One-argument default formatter; every formatter clamps digits to 0–20	Be careful passing a two-argument function as a callback; validate inputs at the edge.

The flicker fix was measured: while a 2,412-character answer streamed in, chart elements changed 0 times. The tooltip fix was tested by hovering every chart on all six views: 78 tooltips, 0 errors, against 8 errors out of 8 hovers before.

11. API reference

All endpoints run on FastAPI at `http://127.0.0.1:8000`. While the backend runs, `/docs` shows interactive documentation where you can try each call.

Method	Path	Input	Returns
GET	`/health`	none	status, dataset loaded
POST	`/upload`	multipart CSV	Full payload for the new file
GET	`/api/datasets`	none	CSV files on the server and the active one
POST	`/api/datasets/load`	`{file_name}`	Loads a file already in uploads/
GET	`/dashboard`	`region`, `channel`, `category`, `season`, `date_from`, `date_to`	Every metric and chart series for that slice
GET	`/api/ai/status`	none	Groq on/off, model in use, last error
POST	`/api/ai/query`	`{query, context_filters, conversation_history, chart_context, stream}`	SSE: meta, delta…, done (JSON if stream is false)
GET	`/api/ai/proactive-insights`	same filters	Up to 6 pills: severity, title, detail, metric, action, impact, query
POST	`/api/ai/scenario-simulate`	`delay_days`, `lead_time_spike_pct`, `demand_shift_pct`, `safety_stock_multiplier`, `price_change_pct`, `price_elasticity_factor`, `context_filters`, `include_narrative`	Baseline vs scenario, regions, exposed segments, delay curve, optional briefing

Older routes (`/sales`, `/seasonality`, `/operations`, `/insights`, `/recommendations`) still answer for compatibility; the frontend no longer calls them.

12. Run, configure, troubleshoot

```
# Terminal 1: backend
cd backend
source .venv/bin/activate
pip install -r requirements.txt      # first time only
uvicorn app:app --reload             # http://127.0.0.1:8000

# Terminal 2: frontend
cd frontend
npm install                          # first time only
npm run dev                          # http://localhost:3000
```

Setting (backend/.env)	Example	Purpose
`GROQ_API_KEY`	`gsk_…`	Turns on live Groq; empty = local engine. Never commit it.
`GROQ_MODEL`	`openai/gpt-oss-120b`	Model to call; falls back automatically if retired
`DEFAULT_DATASET`	`fmcg_clean.csv`	File in uploads/ loaded at start-up
`BACKEND_CORS_ORIGINS`	`http://localhost:3000`	Browser origins allowed to call the API
`VITE_API_BASE_URL` (frontend/.env)	`http://127.0.0.1:8000`	Where the frontend finds the API

Symptom	Likely cause	Fix
Copilot says "local engine" after adding a key	Key in the wrong file, or backend not restarted	Put it in `backend/.env`, restart uvicorn, refresh
"Groq call failed (AuthenticationError…)"	Key wrong or revoked	Create a new key at console.groq.com/keys
"Can't reach the API at …"	Backend not running or wrong port	Start terminal 1; check `VITE_API_BASE_URL`
Dashboard empty after a restart	Data is held in memory only	Pick a file on the Connect screen, or set `DEFAULT_DATASET`
Copilot says there is no Monsoon data	`test_supply_chain.csv` covers 4 days in January	Load `fmcg_clean.csv`
"This view hit an error"	A rendering bug in one view	Try again; the message names the error to report

13. Extend it, and a learning path

Every feature follows one route: compute it in a backend service, expose it in the payload or a route, then draw it in a page.

Add a KPI card

1. In `data_service.py`, add a dict to the list in `_kpis()`: `key`, `label`, `format` (currency, percent, turns, index, days), `value`, `delta`, `status`, `sparkline`.
2. That's all: `KPIGrid` renders every item. Adjust `.grid--kpi` for six columns.

Add a chart

1. Write `_my_series(df)` and add it to `_build_payload()`.
2. In a page: `<SectionCard title="…" inspect={{ data: data.my_series }}>` around a Recharts chart, using `axisProps`, `gridProps`, `COLORS` and `<ChartTooltip rows={…}/>`.
3. Add `isAnimationActive={animate}` from `useMountAnimation()` to each series.

Add an AI feature

1. Add a method on `GroqService` that builds messages with `_messages()` and calls `self._create(…)`; the model fallback comes free.
2. Add a rule-based fallback in `InsightEngine` so it works without a key.
3. Expose it in `routes/ai.py` and call it from `api.js`.

Suggested reading order

1. `data_service.py`: `_preprocess`, `segment_table`, `_kpis`
2. `insight_engine.py`: `build_context`, `proactive_insights`
3. `groq_service.py`: `_messages`, `_create`, `stream_query`
4. `api.js` + `CopilotContext.jsx`: streaming to screen
5. `OperationsPage.jsx`: a full view using every building block

Glossary

SKU	One sellable product variant	**Lead time**	Days from order to delivery
Safety stock	Extra stock held against surprises	**Fill rate**	Share of demand served from stock
OTIF	On-time, in-full delivery rate	**σ (sigma)**	Standard deviation: how spread out values are
SSE	Server-Sent Events: a one-way stream from server to browser	**LLM**	Large language model; Groq hosts them fast
CORS	Browser rule for which sites may call an API	**Payload**	The JSON body of a request or response